

Kolmogorov-Arnold Networks in Low-Data Regimes: A Comparative Study with Multilayer Perceptrons

Farhad Pourkamali-Anaraki

*Department of Mathematical and Statistical Sciences
University of Colorado Denver, CO, USA*

Abstract—Multilayer Perceptrons (MLPs) have long been a cornerstone in deep learning, known for their capacity to model complex relationships. Recently, Kolmogorov-Arnold Networks (KANs) have emerged as a compelling alternative, utilizing highly flexible learnable activation functions directly on network edges, a departure from the neuron-centric approach of MLPs. However, KANs significantly increase the number of learnable parameters, raising concerns about their effectiveness in data-scarce environments. Hence, this paper presents a comprehensive comparative study of MLPs and KANs from both algorithmic and experimental perspectives, with a focus on low-data regimes. We introduce an effective technique for designing MLPs with unique, parameterized activation functions for each neuron, enabling a more balanced comparison with KANs. Using empirical evaluations on two real-world data sets from medicine and engineering, we explore the trade-offs between model complexity and accuracy. Our findings show that MLPs with individualized activation functions achieve higher predictive accuracy with only a modest increase in parameters, especially when the sample size is limited to around one hundred. For example, in a classification problem within additive manufacturing, MLPs achieve a median accuracy of 0.91, significantly outperforming KANs, which only reach a median accuracy of 0.53. In addition, we analyze key factors affecting KAN performance, including the polynomial order and grid size of spline-based activation functions.

Index Terms—Neural Networks, Multilayer Perceptrons, Kolmogorov-Arnold Networks, Activation Function.

I. INTRODUCTION

Multilayer Perceptrons (MLPs) are essential to modern deep learning due to their ability to model intricate nonlinear relationships [1]. MLPs consist of interconnected layers of neurons, forming a network where information flows from input to output. The connections between neurons in adjacent layers are represented by edges, each associated with a weight that determines the strength of the signal passed between them. Within each hidden layer, every neuron performs two tasks. The first one is to calculate a weighted sum of its inputs, aggregating the signals received from the neurons in the previous layer. The second task involves applying a nonlinear activation function to this weighted sum. Without the nonlinear transformations provided by activation functions, neural networks would be limited to linear relationships [2].

Traditionally, MLPs relied on *fixed* activation functions such as the Rectified Linear Unit (ReLU) [3] or the hyperbolic tangent (tanh). Training in this paradigm focuses mainly on adjusting edge weights to reduce error, keeping the activation

functions constant. Recently, there has been a growing interest in parameterized activation functions, which offer greater control during the learning process [4], [5]. These functions contain adjustable parameters, allowing the network to fine-tune its nonlinearities as it trains. This enhanced flexibility can improve performance by tailoring the model's internal transformations to the unique characteristics of the data.

In the pursuit of adaptive activation functions, significant research has focused on parameterizing functions that build upon ReLU, allowing for more nuanced handling of negative inputs. For instance, Leaky ReLU introduces a slight linear slope for negative values [6], while the Exponential Linear Unit (ELU) employs an exponential function in this region [7]. Additionally, the Sigmoid Linear Unit (SiLU), also known as the swish function [8]–[10], has garnered attention for its smooth, non-step-like behavior and its ability to approximate both linear and ReLU characteristics depending on its parameter. Several empirical studies have demonstrated the superior performance of learnable activation functions in large-scale problems involving tens of thousands of samples or more [11].

Recently, Kolmogorov-Arnold Networks (KANs) have been introduced as a novel network architecture [12]. Unlike traditional MLPs, KANs place learnable activation functions on the edges between neurons. This design is rooted in the mathematical principle that multivariate functions can be decomposed into simpler univariate ones using sums [13].

While KANs are designed to utilize more expressive activation functions than standard MLPs with ReLU-like learnable activations, this expressivity comes at the cost of increased parameter count. This raises concerns about their performance in data-sparse scientific and engineering domains, where small sample sizes can hinder effective training. In many fields, such as medicine and engineering, data collection is often limited to a few tens or hundreds of samples due to high costs and time-consuming procedures [14], [15].

Hence, the primary goal of this paper is to provide a comprehensive comparison between MLPs and KANs in low-data scenarios with a few hundred samples. Our key contributions are listed in the following.

- Enabling Fair Comparison: Most deep learning libraries implement MLPs with the same activation function applied to all neurons within a layer. Even when using learnable activation functions, neurons in the same hidden

layer typically share the same set of parameters. To ensure a fair comparison with KANs, we present an effective technique for designing and implementing MLPs where each neuron in a hidden layer has its own distinct, parameterized activation function. This approach can be applied to networks of any depth or width. Furthermore, we implement a parameterized version of the SiLU activation function to ensure that the activation functions used in MLPs are comparable to those used in KANs.

- **Mathematical Connections:** We present a mathematical analysis that elucidates the relationship between MLPs and KANs, showing that KANs can essentially be considered MLPs with activation functions possessing greater flexibility. This analysis underscores the importance of explicitly comparing the number of learnable parameters for both MLPs and KANs in our empirical study.
- **Empirical Evaluation:** We conduct experiments on a simulated data set and two real-world classification problems (cancer detection and 3D printer type prediction) to investigate the trade-offs between model complexity (parameter count) and accuracy in data-limited settings. As a key feature of recently introduced KANs is their ability to stack multiple KAN layers, we specifically examine the impact of network depth on testing accuracy across various data splits. Additionally, our experiments reveal that the piecewise polynomial order of splines significantly influences the performance of KANs.

II. FOUNDATIONS

A widely adopted neural network architecture for tasks involving structured or tabular data is the Multilayer Perceptron (MLP). These networks are composed of sequentially arranged layers with dense interconnections. To be formal, imagine a network with L hidden layers between the input and output layers. The l -th layer consists of N_l neurons. Also, assume that this network takes an input vector $x \in \mathbb{R}^D$, where D is the number of input features. Moreover, the weight matrix and the bias vector can be written as $W^{(l)} \in \mathbb{R}^{N_l \times N_{l-1}}$ and $b^{(l)} \in \mathbb{R}^{N_l}$, for each layer indexed by $l = 1, \dots, L+1$. The predicted output $f(x)$ is then defined from the input x :

$$\begin{aligned} x^{(0)} &= x, \\ x^{(l)} &= g^{(l)}(W^{(l)}x^{(l-1)} + b^{(l)}), \quad l = 1, \dots, L, \\ f(x) &= g^{(L+1)}(W^{(L+1)}x^{(L)} + b^{(L+1)}). \end{aligned} \quad (1)$$

Hence, $f(x)$ takes on a composite or nested form. The number of units in the output layer N_{L+1} and the corresponding activation function $g^{(L+1)}$ depend on the problem at hand. For example, in binary classification problems, a single neuron is typically placed, i.e., $N_{L+1} = 1$, and the Sigmoid activation function is used to find the probability that the data point x belongs to one of the two classes [16]. However, when treating classification problems of more than two classes, the last layer contains one neuron for each class. In this case, we typically employ the Softmax function to find the categorical distribution for all classes. That is, if we have C classes, then the Softmax function accepts a set of C real-valued

numbers $z_1, \dots, z_C$ and converts them into a valid probability distribution by returning $e^{z_c} / \sum_{c'} e^{z_{c'}}$, for $c = 1, \dots, C$ [17].

In contrast, we have more flexibility when choosing activation functions for the L hidden layers, as they produce latent representations. Note that in the equation above, the activation function is applied element-wise. Consequently, the standard implementation of dense layers in popular deep learning libraries such as TensorFlow/Keras applies the same activation function to all neurons within a given layer. In other words, every neuron in a layer employs the same nonlinear function to transmit information to the subsequent layer.

One of the most popular activation functions is the Rectified Linear Unit (ReLU), defined as $\text{ReLU}(z) = \max\{0, z\}$. There are other variants built upon ReLU, such as the Sigmoid Linear Unit (SiLU) [8], also known as the swish function (see [18], [19] for a comprehensive list). A key advantage of $\text{SiLU}(z; \beta) = z/(1 + e^{-\beta z})$ is its smooth, non-step-like behavior. Its behavior can transition between linear (when $\beta = 0$) and ReLU-like (when β is sufficiently large). In many cases, such choices, like the value of β , are predefined hyperparameters and not optimized during training.

A. MLPs with Fully Adaptive Activation Functions

One approach to enhancing the performance and adaptability of MLPs is to treat the parameters within activation functions as trainable during the learning process. This involves incorporating these parameters into the computation graph, allowing the calculation of gradients of the loss function with respect to them. As a result, these activation function parameters, such as the optimal value of β for SiLU, can be learned alongside the network weights.

While several empirical studies [11], [20], [21] have demonstrated the advantages of adaptive activation functions over fixed ones in MLPs, particularly in large-scale data settings like computer vision, it is common to assume that all neurons within a layer share the same activation function parameters. In other words, in Equation (1), $g^{(l)}$ is identical for all N_l neurons in the l -th layer. However, for a fair comparison with KANs, where each connection has its own unique activation function, it is crucial to allow each activation function in MLPs to operate independently, increasing their expressivity.

To achieve this goal, we utilize the built-in *concatenation layer* available in deep learning libraries, enabling us to seamlessly merge the outputs of neurons with distinct activation functions. Let $W_i^{(l)}$ denote the i -th row of the weight matrix $W^{(l)}$, and let $b_i^{(l)}$ represent the i -th element of $b^{(l)}$. Then, each neuron in the l -th layer computes its output as follows:

$$a_i^{(l)} = g_i^{(l)}(W_i^{(l)}x^{(l-1)} + b_i^{(l)}), \quad i = 1, \dots, N_l, \quad (2)$$

where $g_i^{(l)}$ is the activation function specific to the i -th neuron, with its own parameter β_i . The outputs from all N_l neurons are then concatenated to form the output of the l -th layer:

$$x^{(l)} = \text{concatenate}([a_1^{(l)}, \dots, a_{N_l}^{(l)}]) \in \mathbb{R}^{N_l}. \quad (3)$$

This approach offers significant flexibility, allowing us to design and implement MLPs of any depth and width. Each

neuron has its own unique activation function, and the entire process leverages standard automatic differentiation.

B. Kolmogorov-Arnold Networks (KANs)

KANs depart from the traditional MLP architecture by placing activation functions directly on the network's edges (connections between neurons), rather than within the neurons themselves. Consequently, the primary role of each neuron is to simply sum its incoming signals, without applying any additional nonlinearities. This design aims to integrate the nonlinear transformation directly into the weighting mechanism of the edges, rather than separating linear weighting and nonlinear activation as in conventional MLPs.

To formalize this, let us describe the transformation performed by the l -th layer of a KAN, drawing parallels to Equation (1). Recall that the incoming signal $x^{(l-1)}$ has N_{l-1} dimensions and the output of the l -th layer has N_l elements. Let $g_{j,i}^{(l)}(\cdot)$ denote a univariate function that operates on the i -th dimension of $x^{(l-1)}$ to be used for calculating the j -th element of the output. Then, we have the following transformation [22]:

$$x^{(l)} = \begin{bmatrix} g_{1,1}^{(l)}(\cdot) & \cdots & g_{1,N_{l-1}}^{(l)}(\cdot) \\ g_{2,1}^{(l)}(\cdot) & \cdots & g_{2,N_{l-1}}^{(l)}(\cdot) \\ \vdots & & \vdots \\ g_{N_l,1}^{(l)}(\cdot) & \cdots & g_{N_l,N_{l-1}}^{(l)}(\cdot) \end{bmatrix} x^{(l-1)}. \quad (4)$$

Consequently, the transformation performed by each layer remains expressible as a matrix-vector multiplication. However, a crucial aspect of designing KANs lies in the selection of activation functions. For simplicity, let us omit all superscripts and subscripts in the following discussion. It has been proposed that each activation function in Equation (4) can be represented as the weighted sum of SiLU and a spline function:

$$g(x_i) = w_b \text{SiLU}(x_i) + w_s \text{spline}(x_i), \quad (5)$$

where $\text{SiLU}(x_i) = x_i / (1 + e^{-x_i})$ represents the Sigmoid Linear Unit function with $\beta = 1$ and $\text{spline}(x_i) = \sum_k c_k B_k(x_i)$ is a linear combination of B-splines [23], [24]. Thus, the training process involves learning the optimal values of c_k , w_b , and w_s . The flexibility of this formulation depends on two key hyperparameters: the spline order and the grid size.

- The spline order r determines the degree of smoothness and polynomial complexity of each piece in the spline. For instance, $r = 1$ produces piecewise linear functions (similar to ReLU), while higher values such as $r = 3$ yield smoother, more expressive cubic splines.
- The grid size G controls how finely the input domain is partitioned into intervals for spline fitting. A larger grid size results in more local basis functions, allowing the network to capture finer-grained nonlinearities but also increasing the number of parameters.

C. Bridging MLPs and KANs

While KANs clearly utilize activation functions with greater flexibility than standard MLPs employing fixed or parameterized functions such as SiLU, it is worth examining whether

this architectural design represents a fundamentally novel class or merely a reordering of operations. Comparing Equations (1) and (4) reveals that the transformations in both architectures can be expressed in the form of matrix-vector multiplications. However, a key distinction lies in the *order of operations*: MLPs first apply a linear transformation to the input, followed by a nonlinear activation, whereas KANs apply localized nonlinearities to the input first, and then perform a linear combination of the transformed features.

To make this structure explicit, we rewrite the KAN activation function from Equation (5) as:

$$g(x_i) = [w_b \quad w_s c_1 \quad w_s c_2 \quad \cdots \quad w_s c_K] \begin{bmatrix} \text{SiLU}(x_i) \\ B_1(x_i) \\ B_2(x_i) \\ \vdots \\ B_K(x_i) \end{bmatrix}. \quad (6)$$

This highlights that each edge function in a KAN acts as a *weighted sum over a nonlinear feature expansion*, where the basis functions are composed of a fixed SiLU and learned B-splines. To formalize the full layer transformation, we define:

$$\psi_{ji}(x_i) = \begin{bmatrix} \text{SiLU}(x_i) \\ B_1(x_i) \\ \vdots \\ B_K(x_i) \end{bmatrix}, \quad \theta_{ji} = \begin{bmatrix} w_b \\ w_s c_1 \\ \vdots \\ w_s c_K \end{bmatrix}. \quad (7)$$

Then the output of the j -th neuron in a KAN layer can be expressed as:

$$x_j^{(l)} = \sum_{i=1}^{N_{l-1}} \theta_{ji}^\top \psi_{ji}(x_i^{(l-1)}). \quad (8)$$

This expression emphasizes that KANs implement a linear transformation over a *learned, input-dependent nonlinear feature space*, differing from MLPs in structure but not in overall functional form. From a practical standpoint, the central question becomes whether this reordering and the accompanying increase in learnable parameters will yield measurable benefits in accuracy, particularly in low-data regimes.

III. PERFORMANCE EVALUATION: SYNTHETIC DATA

In this section, we commence our comparative study using a two-dimensional simulated data set. This data set encompasses two classes, each consisting of two clusters. Figure 1 illustrates the two data sets central to our analysis: data set A (1,000 samples) and data set B (100 samples). This enables us to directly understand the impact of data size on the performance of KANs, which possess higher degrees of freedom due to the choice of activation functions. For consistency, the default values of grid size 3 and spline order 3 are employed for KANs in this paper, with the exception of Section IV-C where we investigate the impact of hyperparameter adjustments.

Furthermore, to ensure a fair comparison, we utilize the parameterized SiLU activation for the MLP architecture with the learnable parameter β . As previously discussed in Section II-A, each neuron in this MLP architecture possesses its own

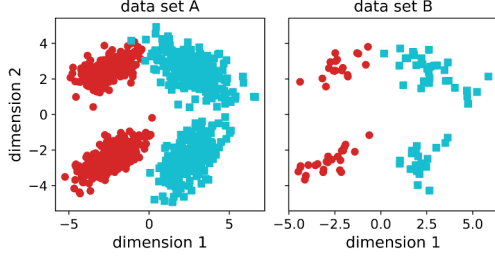


Fig. 1. Visualization of the two simulated data sets with varying sample sizes.

set of trainable parameters, achieved through the concatenation process. For model fitting, we consistently use 50 epochs and a fixed learning rate of 0.05 across all experiments in this paper.

Our comparative analysis focuses on varying network depth L while maintaining a fixed width (number of neurons per hidden layer) to ensure a compact network for evaluating performance in low-data regimes and align with the KAN paper's focus on the KAN layer's stacking potential for deeper models. All experiments use a width of 2 and consider depths of 1, 2, and 3, employing a 70-30 train-test split and measuring classification accuracy on the testing set, *repeated 25 times to account for the impact of data splits and weight initializations*.

For our visualizations, we select violin plots, which provide a richer understanding of the data distribution compared to traditional box plots. The width of each violin at a particular value reflects the concentration of data points around that value, while the vertical axis represents the probability density. Additionally, a tick mark within each violin pinpoints the median of the associated evaluation metric.

As shown in Figure 2(a), both models perform well on data set A due to its separability, resulting in classification accuracies near 1. However, it is noteworthy that both models experience a slight decrease in accuracy as depth increases. Interestingly, this decrease is slightly more pronounced in KANs. For instance, KANs with a depth of 3 reach a minimum accuracy of 0.973, the lowest observed accuracy level. Furthermore, the median accuracy for KANs with depth 3 is 0.990, compared to approximately 0.993 for all MLP depths.

Furthermore, Figure 2(b) reveals a larger performance gap between MLPs and KANs on the smaller data set. While MLPs maintain accuracy levels comparable to the previous data set (e.g., median values near 1), the performance of KANs noticeably degrades in this small-scale scenario. For instance, KANs with depth 3 have a median accuracy of 0.983 and a minimum accuracy of 0.933, a significant drop compared to the minimum of 0.973 observed for data set A.

IV. REAL-WORLD CASE STUDIES

A. Cancer Detection

In this case study, we use the breast cancer wisconsin data set from scikit-learn, which is a well-known benchmark data set in machine learning. It consists of diagnostic information about breast cancer, including input features derived from digitized images of fine needle aspirates (FNAs) of breast

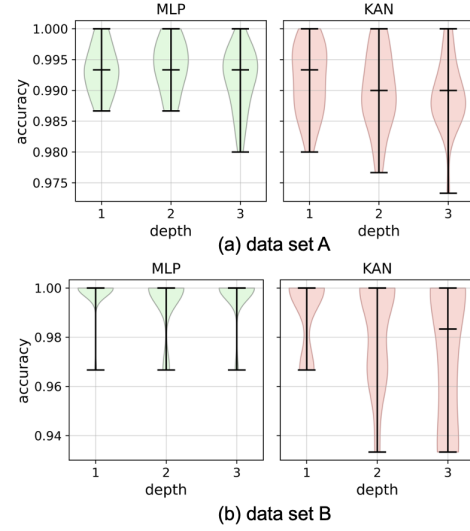


Fig. 2. Accuracy comparison: MLPs vs. KANs on simulated data A and B.

masses. The data set contains 569 samples and each sample is described by 30 real-valued input features, thus $D = 30$. There are 212 malignant and 357 benign samples in the data set, providing a somewhat balanced view of both malignant and benign cases, with the goal of training machine learning models to classify the diagnosis based on the input features.

As detailed in the previous section, we maintain a network width of 2 while varying the depth. To assess classification accuracies, we conduct 25 independent experiments, each with its own train-test split and weight initialization. Figure 3 demonstrates that MLPs significantly outperform KANs. The maximum and minimum accuracy levels for MLPs across all three depths are approximately 0.99 and 0.94, respectively. Therefore, our implementation of MLPs with individually trainable parameters in their SiLU activation functions aligns with existing neural network implementations on this popular data set, which have achieved high accuracies [25].

In contrast, the maximum and minimum accuracy values for KANs are around 0.98 and 0.88, respectively. This wider range of accuracy values, approximately twice that of MLPs, suggests lower overall accuracy and potentially less reliability in their predictions. Similar results are observed when comparing median accuracy levels. For instance, at depth 3, the median accuracy of MLPs is 0.976, while KANs have a substantially lower median value of 0.947. Importantly, MLPs' performance remains fairly consistent across the three depths.

B. 3D Printer Type Prediction

In fused filament fabrication, the mechanical properties of printed parts are influenced not only by printing parameters but also by the specific 3D printer used. Variations in hardware and firmware across printer models lead to differences in material deposition, movement precision, and temperature control, impacting factors like interlayer adhesion and ultimately the final part's strength and surface quality [26].

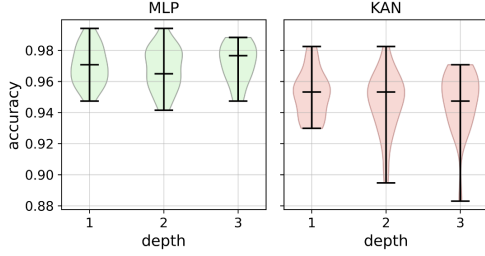


Fig. 3. Evaluating the effect of depth on classification accuracy using the cancer detection data set with 569 samples.

In this section, we evaluate classification models based on MLPs and KANs for identifying the 3D printer used to manufacture a given part. Using a data set comprising tensile properties of parts printed on three different printers (Maker-Bot Replicator 2X, Ultimaker 3, and Zortrax M200) with varying printing parameters [27], our models aim to predict the printer type based on 7 input features: tensile strength, elastic modulus, elongation at break, extrusion temperature, layer height, print bed temperature, and print speed. This approach seeks to capture the subtle relationships between printing process, part properties, and the specific printer used, which can be viewed as a three-class classification problem.

Figure 4 presents classification accuracy results across 25 repetitions. Overall, MLPs demonstrate strong performance across all three depth values. For instance, with a depth of 3, the maximum, median, and minimum accuracies are 1, 0.906, and 0.844, respectively. These results highlight the effectiveness of MLPs, especially considering this is a three-class classification problem where a random classifier would yield accuracies closer to 0.333, significantly lower than our observed results. On the other hand, we observe a more pronounced accuracy drop in KANs compared to the previous data set. While KANs can achieve accuracies at or near 1 in some repetitions, there are instances where accuracy falls below 0.333, the baseline for a random classifier. Fortunately, the median values for KANs at depths 1, 2, and 3 are 0.531, 0.406, and 0.406, respectively, all surpassing the baseline.

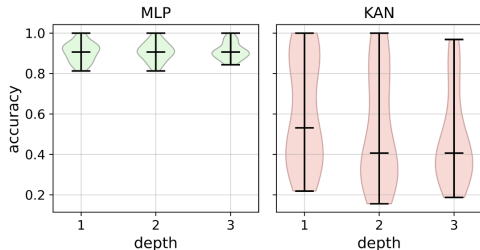


Fig. 4. Evaluating the effect of depth on classification accuracy: a comparative study using the 3D printer type prediction data set with 104 samples.

C. Dependence of KANs on the Order and Grid Size

In this section, we delve deeper into the performance of KANs on these two real-world data sets to understand how the complexity of the activation functions influences their

behavior. Each activation function in KANs is parameterized as a B-spline, and a crucial hyperparameter is the polynomial order of these splines. An order of 1 results in an activation function similar to ReLU, i.e., a piecewise linear function, while higher orders provide increasing degrees of nonlinearity.

The default spline order in the KAN implementation is 3. To gain further insight into the impact of this choice in low-data regimes, we consider networks of depth 2 and width 2, but we vary the spline order from 1 to 5. Figure 5(a) reveals an interesting trend: in most cases, the accuracy of KANs on the cancer detection data set decreases as the spline order increases. The highest median accuracy is achieved at order 2 (0.959), while the lowest is at order 5 (0.929). However, MLPs with parameterized swish activations per neuron still outperform the best KAN configuration in this experiment. The median accuracy for MLPs with depth 2 reaches 0.965, while maintaining an order of magnitude fewer learnable parameters. This demonstrates that MLPs with individualized parameterized activations can achieve higher accuracy with significantly fewer parameters.

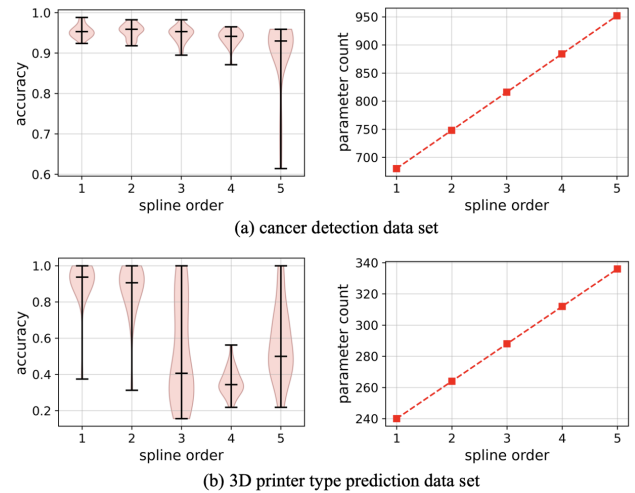


Fig. 5. Investigating the impact of spline order on both the accuracy of KANs and the number of learnable parameters, utilizing two real-world data sets: (a) cancer detection and (b) 3D printer type prediction.

V. CONCLUSION AND FUTURE DIRECTIONS

While Kolmogorov-Arnold Networks (KANs) offer an intriguing alternative to Multilayer Perceptrons (MLPs) by replacing linear weights with highly expressive activation functions, our findings highlight their notable performance degradation in low-data regimes compared to MLPs. As our algorithmic comparison revealed, the primary strength of KANs lies in their choice of activation functions because they implicitly incorporate a linear weighting mechanism similar to MLPs. This underscores the critical role of activation function complexity in neural network performance. While complex activation functions with greater flexibility might seem like the obvious choice, our findings suggest that simpler alternatives

such as the SiLU can provide adequate capacity for some practical applications, particularly in data-scarce scenarios.

Additionally, our research has revealed that individually parameterized neurons within hidden layers can derive advantages from utilizing independent, individualized activation functions. This approach does not compromise accuracy and opens new avenues for enhancing the predictive power of smaller networks on small-scale data sets. Implementing such MLPs with fully adaptive activation functions is straightforward in popular deep learning libraries like TensorFlow/Keras and PyTorch using concatenation layers. It is imperative to include these MLPs in future comparative studies, as existing benchmarks focusing on fixed-shape activation functions, e.g., [28], may not provide a fair comparison. This paves the way for exploring novel activation functions that offer controlled nonlinear transformations for analyzing complex data.

Several recent works have explored activation functions beyond splines in KANs, which can also be applied to MLPs with individualized activations. For example, wavelet functions can capture both high-frequency and low-frequency components of input data [29]. Another promising direction is developing adaptive algorithms for selecting activation functions per neuron in MLPs, considering a predefined space including splines, wavelets, Chebyshev polynomials, and others [30], [31]. Furthermore, a recent survey paper [32] offers insights into integration with diverse architectures, such as convolutional, recurrent, and transformer-based models, while also addressing key aspects, e.g., efficiency, interpretability, and scalability [33], [34].

REFERENCES

- [1] M. Raghu, B. Poole, J. Kleinberg, S. Ganguli, and J. Sohl-Dickstein, "On the expressive power of deep neural networks," in *International Conference on Machine Learning*, pp. 2847–2854, 2017.
- [2] S. Hayou, A. Doucet, and J. Rousseau, "On the impact of the activation function on deep neural networks training," in *International Conference on Machine Learning*, pp. 2672–2680, 2019.
- [3] R. Vershynin, "Memory capacity of neural networks with threshold and rectified linear unit activations," *SIAM Journal on Mathematics of Data Science*, vol. 2, no. 4, pp. 1004–1033, 2020.
- [4] F. Agostinelli, M. Hoffman, P. Sadowski, and P. Baldi, "Learning activation functions to improve deep neural networks," *arXiv preprint arXiv:1412.6830*, 2014.
- [5] K. Lee, J. Yang, H. Lee, and J. Hwang, "Stochastic adaptive activation function," *Advances in Neural Information Processing Systems*, pp. 13787–13799, 2022.
- [6] A. Maniopoulos and N. Mitianoudis, "Learnable leaky ReLU (LeLeLU): An alternative accuracy-optimized activation function," *Information*, vol. 12, no. 12, p. 513, 2021.
- [7] L. Trottier, P. Giguere, and B. Chaib-Draa, "Parametric exponential linear unit for deep convolutional neural networks," in *International Conference on Machine Learning and Applications*, pp. 207–214, 2017.
- [8] P. Ramachandran, B. Zoph, and Q. Le, "Searching for activation functions," *arXiv preprint arXiv:1710.05941*, 2017.
- [9] M. Tanaka, "Weighted sigmoid gate unit for an activation function of deep neural network," *Pattern Recognition Letters*, vol. 135, pp. 354–359, 2020.
- [10] M. Kaytan, I. Aydılek, and C. Yeroğlu, "Gish: a novel activation function for image classification," *Neural Computing and Applications*, vol. 35, no. 34, pp. 24259–24281, 2023.
- [11] A. Apicella, F. Donnarumma, F. Isgro, and R. Prevete, "A survey on modern trainable activation functions," *Neural Networks*, vol. 138, pp. 14–32, 2021.
- [12] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Hou, and M. Tegmark, "KAN: Kolmogorov-Arnold Networks," *arXiv preprint arXiv:2404.19756*, 2024.
- [13] J. Schmidt-Hieber, "The Kolmogorov–Arnold representation theorem revisited," *Neural Networks*, vol. 137, pp. 119–126, 2021.
- [14] G. Qi and J. Luo, "Small data challenges in big data era: A survey of recent progress on unsupervised and semi-supervised methods," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 4, pp. 2168–2187, 2020.
- [15] F. Pourkamali-Anaraki, T. Nasrin, R. Jensen, A. Peterson, and C. Hansen, "Adaptive activation functions for predictive modeling with sparse experimental data," *Neural Computing and Applications*, 2024.
- [16] F. Pourkamali-Anaraki, T. Nasrin, R. Jensen, A. Peterson, and C. Hansen, "Evaluation of classification models in limited data scenarios with application to additive manufacturing," *Engineering Applications of Artificial Intelligence*, vol. 126, p. 106983, 2023.
- [17] J. Ren, C. Yu, X. Ma, H. Zhao, and S. Yi, "Balanced meta-softmax for long-tailed visual recognition," *Advances in Neural Information Processing Systems*, vol. 33, pp. 4175–4186, 2020.
- [18] S. Dubey, S. Singh, and B. Chaudhuri, "Activation functions in deep learning: A comprehensive survey and benchmark," *Neurocomputing*, 2022.
- [19] A. Jagtap and G. Karniadakis, "How important are activation functions in regression and classification? a survey, performance comparison, and future directions," *Journal of Machine Learning for Modeling and Computing*, vol. 4, no. 1, 2023.
- [20] K. Biswas, S. Kumar, S. Banerjee, and A. Pandey, "Erfact and pserf: Non-monotonic smooth trainable activation functions," in *AAAI Conference on Artificial Intelligence*, pp. 6097–6105, 2022.
- [21] S. Kiliçarslan and M. Celik, "Parametric RSigELU: a new trainable activation function for deep learning," *Neural Computing and Applications*, vol. 36, no. 13, pp. 7595–7607, 2024.
- [22] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljacic, T. Hou, and M. Tegmark, "KAN: Kolmogorov–Arnold Networks," in *The Thirteenth International Conference on Learning Representations*, 2025.
- [23] M. Unser, "A representer theorem for deep neural networks," *Journal of Machine Learning Research*, vol. 20, no. 110, pp. 1–30, 2019.
- [24] P. Bohra, J. Campos, H. Gupta, S. Aziznejad, and M. Unser, "Learning activation functions in deep (spline) neural networks," *IEEE Open Journal of Signal Processing*, vol. 1, pp. 295–309, 2020.
- [25] M. Alshayegi, H. Ellethy, S. Abed, and R. Gupta, "Computer-aided detection of breast cancer on the wisconsin dataset: An artificial neural networks approach," *Biomedical Signal Processing and Control*, vol. 71, p. 103141, 2022.
- [26] T. Nasrin, F. Pourkamali-Anaraki, and A. Peterson, "Application of machine learning in polymer additive manufacturing: A review," *Journal of Polymer Science*, vol. 62, no. 12, pp. 2639–2669, 2024.
- [27] D. Braconnier, R. Jensen, and A. Peterson, "Processing parameter correlations in material extrusion additive manufacturing," *Additive Manufacturing*, vol. 31, p. 100924, 2020.
- [28] E. Poeta, F. Giobergia, E. Pastor, T. Cerquitelli, and E. Baralis, "A benchmarking study of kolmogorov-arnold networks on tabular data," *arXiv preprint arXiv:2406.14529*, 2024.
- [29] Z. Bozorgasl and H. Chen, "Wav-kan: Wavelet kolmogorov-arnold networks," *arXiv preprint arXiv:2405.12832*, 2024.
- [30] Z. Yang, J. Zhang, X. Luo, Z. Lu, and L. Shen, "Activation space selectable kolmogorov-arnold networks," *arXiv preprint arXiv:2408.08338*, 2024.
- [31] K. Shukla, J. Toscano, Z. Wang, Z. Zou, and G. Karniadakis, "A comprehensive and fair comparison between MLP and KAN representations for differential equations and operator networks," *Computer Methods in Applied Mechanics and Engineering*, vol. 431, p. 117290, 2024.
- [32] S. Somvanshi, S. Javed, M. Islam, D. Pandit, and S. Das, "A survey on kolmogorov-arnold network," *ACM Computing Surveys*, 2024.
- [33] A. Howard, B. Jacob, and P. Stinis, "Multifidelity Kolmogorov–Arnold networks," *Machine Learning: Science and Technology*, vol. 6, no. 3, p. 035038, 2025.
- [34] A. Jamali, S. Roy, D. Hong, B. Lu, and P. Ghamisi, "How to learn more? exploring Kolmogorov–Arnold networks for hyperspectral image classification," *Remote Sensing*, vol. 16, no. 21, p. 4015, 2024.